Labo 13 — Services et Ingress : exposer

Théorie — pourquoi l'IP d'un Pod ne sert à rien, comment un nom stable se branche sur des Pods qui changent, et par quelles portes le trafic entre dans le cluster : ClusterIP, NodePort, LoadBalancer, Ingress.

Objectifs

- Comprendre ce qu'un **Service** ajoute aux Pods : une IP virtuelle, un nom DNS, une répartition.
- Distinguer `ClusterIP`, `NodePort`, `LoadBalancer`, et savoir lequel utiliser.
- Lire les **EndpointSlices** et comprendre le lien avec la readiness.
- Exposer le front par un **Ingress** et comprendre le rôle de l'Ingress controller (Traefik).
- Retrouver, dans tout cela, le DNS et le reverse proxy des labos 07 et 09.

---

1. Le problème : des IP qui n'arrêtent pas de changer

Au labo 12, chaque rollout a remplacé les Pods, et chaque Pod neuf a reçu une nouvelle IP `10.42.0.x`. Un front qui aurait codé l'adresse d'un Pod de l'API serait tombé à la première mise à jour. Pire : avec trois réplicas, laquelle appeler ? Dans le labo pratique, nginx démarré avant tout Service échoue net : `host not found in upstream "api"` — rien dans le cluster ne porte ce nom.

Le **Service** résout les deux problèmes : il fournit un **nom** et une **IP stables** (l'IP de cluster, `10.43.x.x` chez k3s, jamais réattribuée tant que le Service existe) devant un **ensemble de Pods** choisis par label, et répartit les connexions entre eux. Les Pods vont et viennent ; le Service reste.

> **→ RAPPEL DOCKER**
> Au labo 07, le réseau user-defined donnait un nom DNS à chaque **conteneur** ; au labo 09, Compose le faisait pour chaque **service**, et `--scale api=2` donnait deux adresses au même nom. Le Service Kubernetes est la version aboutie de cette idée : un nom, une IP virtuelle, une liste d'adresses réelles maintenue à jour par le cluster, et une répartition faite par le nœud plutôt que par le client.

2. Anatomie d'un Service

```
apiVersion: v1
kind: Service
metadata:
  name: api                 # le nom DNS
spec:
  selector:
    app: api                # les Pods visés, par label
  ports:
    - port: 8080            # port du Service (celui que les clients appellent)
      targetPort: 8080      # port du conteneur
```

Trois mécanismes se cachent derrière ce fichier. **Le DNS** : CoreDNS crée l'entrée `api.lab13.svc.cluster.local` → IP de cluster ; depuis le même namespace, `api` suffit grâce au `search` du `/etc/resolv.conf` des Pods. **Les EndpointSlices** : un contrôleur recense en permanence les IP des Pods qui portent le label **et** sont `Ready` ; `kubectl get endpointslices` les

montre, et c'est la liste réelle derrière l'IP virtuelle. **kube-proxy** : sur chaque nœud, il programme des règles (iptables ou nftables) qui réécrivent toute connexion vers l'IP de cluster en une connexion vers l'un des endpoints, au hasard.

> **→ RÉSEAU**
> L'IP de cluster n'est attachée à **aucune interface** : aucun processus n'écoute dessus, `ping` ne répond pas. C'est une adresse purement virtuelle que les règles de **NAT** de kube-proxy transforment à la volée en adresse de Pod, sur le nœud d'où part le paquet. La répartition est donc faite par connexion, au niveau 4 (TCP), sans notion de requête HTTP — un client qui garde sa connexion ouverte reste sur le même Pod.

Deux conséquences pratiques. Un Pod **non prêt** (readiness en échec, labo 12) disparaît des EndpointSlices : le Service cesse de lui envoyer du trafic sans qu'il soit tué — c'est tout l'intérêt de la readiness. Et un Service **sans selector** n'a pas d'endpoints : utile pour nommer une base externe au cluster (avec un EndpointSlice écrit à la main), piège quand le label est mal orthographié (`kubectl get endpointslices` vide = premier réflexe).

3. Les types de Service : par où entre le trafic

Type	Joignable depuis	Comment	Quand
ClusterIP (défaut)	l'intérieur du cluster	IP virtuelle + DNS	tout service interne : `api`, `db`
NodePort	l'extérieur, via un port de **chaque nœud** (30000-32767)	`IP-du-nœud:30080` → Service	test, lab, accès sans équilibreur
LoadBalancer	l'extérieur, via une IP fournie par l'infrastructure	un équilibreur externe → NodePort → Service	exposition publique sur le cloud
ExternalName	l'intérieur	un alias DNS vers un nom externe	nommer un service hors cluster

Chaque type **inclut** le précédent : un NodePort a aussi une ClusterIP, un LoadBalancer a aussi un NodePort. Le NodePort montre une limite : il faut connaître l'adresse d'un nœud, et le port est bizarre. Le LoadBalancer la corrige en demandant à l'infrastructure une adresse « propre » — sur le cloud, c'est un équilibreur facturé ; sur un cluster nu, l'`EXTERNAL-IP` reste `<pending>` pour toujours, faute de fournisseur.

> **K3S**
> k3s livre **ServiceLB** (anciennement Klipper) : pour chaque Service `LoadBalancer`, il lance un Pod `svclb-<nom>` dans `kube-system` qui ouvre le port demandé **sur le nœud** et le relaie ; l'`EXTERNAL-IP` devient l'IP du nœud. Deux Services ne peuvent donc pas demander le même port (le second reste `<pending>`), et c'est pourquoi Traefik occupe déjà les ports 80 et 443. Sur EKS/AKS/GKE, le même manifeste produit un vrai équilibreur avec une IP publique : le YAML ne change pas, la facture oui.

> **→ WINDOWS / WSL**
> Un NodePort ou un port ServiceLB est joignable depuis Windows sur `http://<ip-de-la-distribution>:30080` (`hostname -I` dans WSL), et le plus souvent sur `http://localhost:30080` grâce au forwarding WSL — pour les ports ouverts par k3s, qui tourne en root, la restriction des ports < 1024 du labo 07 ne s'applique pas : Traefik écoute bien sur **80**. `kubectl port-forward` reste le chemin le plus sûr pour un test rapide : il ne dépend d'aucun type de Service.

4. L'Ingress : une seule porte pour tout le HTTP

Exposer chaque front par un LoadBalancer coûte une adresse (et un équilibreur) par application. L'**Ingress** mutualise : un seul point d'entrée HTTP/HTTPS, et des **règles** qui routent par nom d'hôte (`app.example.com`) et par chemin (`/api/`) vers des Services. L'objet Ingress n'est qu'une **description** ; c'est un **Ingress controller** — un reverse proxy qui tourne dans le cluster et lit ces objets — qui fait le travail : Traefik chez k3s, ingress-nginx, HAProxy, ou l'équilibreur applicatif du cloud.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

Le trafic suit alors : navigateur → port 80 du nœud (ServiceLB de Traefik) → Pod Traefik → Service `web` → Pod nginx → (pour `/api/`) Service `api` → Pod de l'API. Deux reverse proxies en cascade, Traefik puis nginx : c'est courant, et c'est pourquoi beaucoup d'équipes font router `/api/` directement par l'Ingress vers le Service `api`, en retirant le proxy de nginx.

> **→ HTTP**
> L'Ingress travaille au niveau 7 : il lit l'en-tête `Host` et le chemin de chaque requête, peut terminer le **TLS** (certificats, souvent automatisés par cert-manager), réécrire des chemins, ajouter des en-têtes (`X-Forwarded-For`, précieux après le labo 07), limiter le débit. Un Service, lui, ne voit que des connexions TCP. Le successeur de l'Ingress, la **Gateway API**, généralise ces règles ; Traefik la prend en charge, mais l'Ingress reste le standard déployé partout.

5. Le diagnostic en quatre questions

Quand « ça ne répond pas », remonter la chaîne : (1) **les Pods sont-ils prêts ?** `kubectl get pods` — `0/1` = readiness ; (2) **le Service a-t-il des endpoints ?** `kubectl get endpointslices` vide = selector faux ou Pods non prêts ; (3) **le nom résout-il ?** `kubectl exec … -- wget -qO- http://api:8080/` depuis un Pod du même namespace ; (4) **la porte d'entrée est-elle la bonne ?** `kubectl get svc` pour le type et le port, `kubectl get ingress` pour l'adresse et les règles, `kubectl describe ingress` pour les backends. Un `503` de Traefik signifie presque toujours « le Service visé n'a aucun endpoint prêt ».

6. En entreprise

- **Un Service ClusterIP par Deployment**, du même nom, dans le même manifeste : `api` et `db` ne sont jamais exposés hors du cluster.

- **Un seul Ingress controller** pour tout le cluster, géré par l'équipe plateforme ; chaque application déclare ses règles Ingress (ou Gateway) avec un nom d'hôte et un certificat TLS.
- **Le front appelle l'API par un chemin relatif** (`/api/`), routé par l'Ingress ou par nginx : même origine, pas de CORS (labo 07), et l'URL ne change pas d'un environnement à l'autre.
- **Les NodePorts** restent sur le poste et en lab ; le `LoadBalancer` est réservé aux rares services TCP non HTTP (bases répliquées, brokers), et facturé.

À retenir

- Un Service = un nom DNS + une IP de cluster stable + une liste d'endpoints maintenue par label et readiness + une répartition par kube-proxy.
- `api` résout dans le namespace, `api.lab13.svc.cluster.local` partout ; sans Service, nginx meurt sur `host not found in upstream`.
- ClusterIP pour l'interne, NodePort pour tester, LoadBalancer pour une IP externe (ServiceLB chez k3s, équilibreur cloud ailleurs), Ingress pour tout le HTTP.
- Un Pod non prêt sort des EndpointSlices ; un selector faux donne un Service sans endpoints et un `503`.
- L'Ingress décrit des règles ; l'Ingress controller (Traefik) les applique sur le port 80/443 du cluster.
- Diagnostic : Pods prêts → endpoints → DNS → porte d'entrée.

Vocabulaire

Service : nom et IP stables devant des Pods sélectionnés par label. — **ClusterIP** : IP virtuelle interne au cluster. — **NodePort** : port ouvert sur chaque nœud (30000-32767). — **LoadBalancer** : IP externe fournie par l'infrastructure (ServiceLB chez k3s). — **EndpointSlice** : liste des IP de Pods prêts derrière un Service. — **kube-proxy** : règles NAT par nœud qui réalisent le Service. — **CoreDNS** : DNS du cluster (`<svc>.<ns>.svc.cluster.local`). — **Ingress** : règles de routage HTTP par hôte et chemin. — **Ingress controller** : reverse proxy qui applique les Ingress (Traefik). — **Gateway API** : successeur de l'Ingress.